

CSEN 296B-2 | Week 8.1 Lab Submission

AstraNotes Collaborative Git Workflow

Student Name: Atishay Jain | **Date:** May 19, 2026 | **Project:** AstraNotes | **Technical Path:** Python
Repository: github.com/Atishay8192261/astranotes (private) | **Pull Request:** #1 (merged)

Collaboration Log

AstraNotes is a solo build, so I ran the rubric as a *two-hat* simulation: author hat on the feature branch, reviewer hat on the PR. The point was not to perform the motions but to make the change actually reviewable - branch, commit, PR, critique, merge - and to write the kind of review I would want from a teammate, not the kind that just clicks approve.

I picked the change before opening the branch. Candidates were search improvement, settings/UI polish, and test cleanup. I chose **search** because it is the largest unimplemented behavior in the committed scope (US-06 / FR-07), it has real edge cases worth reviewing (case folding, decrypt-before-match, undecryptable notes, empty corpus), and it gives me something honest to flag in the review instead of rubber-stamping a trivial diff. Test cleanup would have been too thin to write a useful PR summary about; UI polish would not have been visually verifiable without Tk in this environment.

Branch and PR Workflow Summary

Step	What I did	Why
1	<code>git checkout main && git pull</code>	Start from a known-clean baseline; never branch from stale main.
2	<code>git checkout -b feature/search-notes</code>	One branch, one idea. Branch name encodes intent (US-06).
3	Two focused commits on the branch	Service tier first, then controller/GUI wiring. Each commit small enough that a revert peels one off without taking the other.
4	<code>pytest -q</code>	46 pass locally before push. Pushing red wastes a reviewer's time.
5	<code>git push -u origin feature/search-notes</code>	Remote-tracked from the start; no orphaned branch later.
6	<code>gh pr create</code>	Structured body: summary, traceability, test inventory, deliberate out-of-scope. The PR body is what survives in history.
7	<code>gh pr review --comment</code>	Reviewer hat. Did not approve until I had named at least one thing worth changing.
8	<code>gh pr merge --squash --delete-branch</code>	Squash keeps main history linear; delete branch keeps the remote tidy.

Branch Purpose

feature/search-notes exists to realize the US-06 keyword-search slice end-to-end, from *NoteManager* through the controller seam to a live search entry above the sidebar, with no other scope.

Commit Messages (as they appear in the branch)

Add `NoteManager.search_notes` (FR-07/US-06)

```
Case-insensitive substring match across titles and bodies. Private
notes decrypt before matching; undecryptable notes are skipped with
an error logged (SPR-01/SPR-02). Empty keyword returns empty list.
```

Wire search into controller and GUI sidebar (US-06)

Adds `NotesController.search_notes()` passthrough and a search entry above the sidebar; live key-release filtering. Empty keyword falls back to `list_notes` (FR-08 ordering preserved). +1 controller test.

Both messages name what changed and which requirement it traces to. Neither rambles. The squash commit on main carries the PR number (#1) for trace-back.

PR Summary (the one I wrote on PR #1)

Title: Add keyword search (US-06 / FR-07)

Summary

- Adds `NoteManager.search_notes(keyword)` - case-insensitive substring match across titles and bodies (FR-07).
- Private notes are decrypted before matching; undecryptable notes are skipped with an error logged, preserving SPR-01 (no plaintext leak) and SPR-02 (no traceback / no path exposure).
- Wires the behavior up through `NotesController.search_notes()` and a live search entry above the GUI sidebar. Empty keyword falls back to `list_notes()` so FR-08 ordering is preserved.

Traceability

ID	How this PR closes / honors it
US-06	First realization slice for Search Notes by Keyword.
FR-07	Case-insensitive, plain substring, decrypted private notes included, failed decryptions excluded, empty collection returns empty list.
FR-08	Search reuses <code>list_notes()</code> so the existing modified_at-desc ordering test still proves the property.
SPR-01 / SPR-02	Decryption-failure path exercised by <code>test_search_notes_skips_undecryptable_private_note</code> ; no plaintext leak; no path/traceback exposed.

Tests: 40 -> 46. Five new service-tier tests + one new controller test. Full suite green (*46 passed in 0.10s*), no Tk required.

Out of scope (deliberate): NFR-01 benchmark, regex/boolean operators, persisted recent-search history.

Example of Review Feedback (the review I wrote on PR #1)

I approved with two follow-ups instead of stamping a blind LGTM. The most useful thing a review can do is say *what the diff is silent about*. Excerpt of the review body:

1. Silent skip on decryption failure is logged but not surfaced to the user (medium severity).

`search_notes()` delegates to `list_notes()`, which already drops un-decryptable private notes with a `logger.error`. That preserves SPR-01 (no plaintext leak), but a user searching for a term that *would have* matched an un-decryptable note sees an empty result with no indication that something was skipped. Consistent with current `list_notes()` behavior, so not a regression - but when key-rotation lands (ADR-005 still Pending), search should surface a 'N notes could not be searched' hint via the status bar. Refactor candidate, not a merge blocker.

2. GUI search rebuilds the entire sidebar on every KeyRelease (low severity). For the 500-note NFR-01 target this is borderline - a full `list_notes()` scan with N Fernet decryptions happens per keystroke. Not blocking because (a) NFR-01 is out of scope for this PR, (b) the substring match itself is microsecond-cheap, (c) decryption dominates only when many private notes exist. A ~150 ms debounce or an in-memory plaintext cache would fix it if it becomes a UX problem.

Both findings are tracked as refactor notes. Neither blocks the merge because the merged behavior satisfies FR-07 as written. That distinction - *consistent-with-spec but worth a follow-up* vs. *blocks merge* - is the call a reviewer has to make every time, and I tried to make it explicitly.

The non-feedback half of the review matters too: I called out what was already done well so the next change doesn't accidentally undo it. The decryption-failure test uses two real *PrivacyService* instances with different keys instead of a mock; that matches the project's 'no mocked storage' convention from Week 7.2 and would be easy to regress to a mock under time pressure.

Merge Decision and Merge-Readiness Note

Decision: merged via squash-merge into *main* on 2026-05-19.

Why it was safe to merge:

- Behavior matches FR-07 exactly; no committed acceptance criterion was bent.
- Full test suite green (40 -> 46), including the high-risk SPR-01 decryption-failure path. The five new service tests use *tmp_path* + a real *JsonFileRepository* + two real *PrivacyService* instances - not mocked.
- The change is additive. *search_notes()* is a new public method; the GUI search entry is a new widget. No existing call path changed signature or semantics. *list_notes()* and FR-08 ordering are untouched.
- SPR-01 is provably preserved: the only place private bodies appear in plaintext is in-memory inside *list_notes()*, which is exactly how the codebase already handles list. No new disk write path was introduced.

Why a thoughtful reviewer might still hold the merge (and I chose not to): the keystroke-rate sidebar rebuild is a latent NFR-01 concern, accepted because NFR-01 is explicitly out of scope and was not enforced anywhere else either; the silent-skip behavior is a real UX gap, held because closing it requires a controller-level result type that does not belong inside this scoped slice.

Refactor Note (post-merge follow-up)

Two refactor candidates surfaced during review. Both are intentionally deferred, with the reason explicit so they don't decay into 'we'll get to it':

1. Surface skipped-during-search count. Change *NoteManager.search_notes()* (and the controller passthrough) to return (*matches*, *skipped_count*), so the GUI status bar can show 'N notes could not be searched' when a decryption failure silently dropped a candidate. Right *after* ADR-005 (Pending) is resolved and key-rotation becomes a real path; doing it now would be designing for a code path that doesn't yet exist.

2. Debounce the keystroke search. Bind the search entry to a ~150 ms debounce timer instead of `<KeyRelease>` direct. Trivial change, deferred until either a real performance complaint shows up or the NFR-01 benchmark lands - whichever is first.

What I will not refactor: the search algorithm itself. Plain lowercased substring match is exactly what FR-07 specifies, and the temptation to 'improve' it to regex / fuzzy / weighted ranking would be scope creep against an accepted requirement - that goes through the backlog, not through a code change.

How AI Helped, and What I Accepted / Changed / Rejected

Accepted. Copilot suggested the explicit *keyword.strip()* empty-guard and the test pair covering both `"` and `'`. Obvious in retrospect but easy to miss; I kept them because they are cheap and they make the FR-07 'empty collection returns empty list' promise hold for the *empty keyword* edge too. Copilot also suggested using two distinct *PrivacyService* instances for the undecryptable-search test rather than monkeypatching *decrypt* to raise - matches the convention from Week 7.2, so I took it as-is.

Changed. Copilot's first draft of *search_notes* ran a separate decrypt loop inside the search method, duplicating the *list_notes()* decrypt-and-skip logic. I rewrote it to delegate to *list_notes()* instead. The rewrite is shorter, but the real reason is that it keeps the FR-07 / FR-08 contract defined in exactly one place. Two copies of decrypt-and-skip is two places to leak plaintext when the next change goes in.

Rejected. Copilot proposed a 'search by tag' extension and a regex-mode toggle. Both are out of scope per FR-07 ('plain string matching, not regex') and would violate the Working Agreement's no-scope-creep rule. I also rejected its suggestion to add a self-review comment on the PR that just listed the changes - reviews exist to surface what the diff is silent about, and a recap of what the diff already shows is review theater.

On the review itself. I drafted the review note manually first, then asked Copilot to critique it for hand-wavy claims. It correctly flagged that my original phrasing on the keystroke-rebuild concern said 'this might be slow' without quantifying *when* it would be slow. I tightened that paragraph to name the NFR-01 target and the dominating cost (Fernet decryption), which is what makes it actionable instead of a vague worry. The keep/change/reject decisions and the merge call are mine.

The honest limit. AI helped me widen the candidate list and tighten my review prose. It did not decide what to merge, what to defer, or what risk profile to accept. Those are exactly the judgments the rubric asks for, and they have to be mine.